

Pandas 101 : A Comprehensive Guide to Mastering Data Analysis with Python's Pandas Library



Niraj Tiwari

Follow

7 min read · Jan 9, 2024



91



1. Introduction to Pandas

Pandas is an open-source library in Python used for data manipulation and analysis. It offers data structures and operations for manipulating numerical tables and time series. It's particularly well-suited for handling structured data, i.e., data that is organized into tables.

Series and DataFrames: The two primary data structures in Pandas.

- A **Series** is a one-dimensional array-like structure.
- A **DataFrame** is a two-dimensional, size-mutable, and potentially heterogeneous tabular data structure.

Example 1: Creating a Series

```
import pandas as pd
s = pd.Series([1, 3, 5, 7, 9])
print(s)
```

Example 2: Creating a DataFrame

```
data = {'Name': ['John', 'Anna', 'Peter', 'Linda'],
        'Age': [28, 34, 29, 32]}
df = pd.DataFrame(data)
print(df)
```

```
# Creating a DataFrame from a dictionary.
import pandas as pd
```

```
data = {  
    'Name': ['Alice', 'Bob', 'Charlie'],  
    'Age': [25, 30, 35],  
    'City': ['New York', 'Paris', 'London']  
}  
  
df = pd.DataFrame(data)  
print(df)
```

2. Data Importing and Exporting

Pandas supports various data formats for importing and exporting, such as CSV, Excel, and SQL databases.

Reading from CSV

```
df = pd.read_csv('path/to/your/file.csv')  
print(df.head())
```

Writing data to CSV

```
df.to_csv('path/to/save/file.csv', index=False)
```

For more on importing/exporting: [Pandas IO Tools Documentation](#)

3. Basic Data Operations

These operations involve viewing, selecting, indexing, and slicing data.

Viewing Data

```
# To look at the first few rows of a DataFrame.  
print(df.head()) # First five rows
```

Selecting a Column

```
# To access a column in the DataFrame.  
ages = df['Age']  
print(ages)  
print(df['Name']) # Displays the 'Name' column
```

Slicing Rows

```
subset = df[0:2] # First two rows  
print(subset)
```

Filtering Data: Selecting rows based on a condition.

```
filtered_df = df[df['Age'] > 30] # Rows where age is greater than 30  
print(filtered_df)
```

Adding Columns: You can add new columns to a DataFrame.

```
df['AgeInTenYears'] = df['Age'] + 10  
print(df)
```

4. Data Cleaning

Data cleaning involves handling missing data, data type conversion, and renaming/replacing values.

Handling Missing Data

```
df.fillna(0, inplace=True) # Replace missing values with 0
```

Data Type Conversion

```
df['Age'] = df['Age'].astype(float)
```

Renaming Columns

```
df.rename(columns={'Age': 'AgeYears'}, inplace=True)
```

5. Basic Data Analysis

Pandas provides functions to perform basic descriptive statistics and aggregations.

Descriptive Statistics

```
# Quickly get a statistical summary of the data.  
print(df.describe())
```

Aggregation

```
print(df['Age'].mean()) # Average age
```

6. Basic Data Visualization

Pandas can integrate with Matplotlib and Seaborn for basic plotting.

Line Plot

```
df.plot(kind='line')
```

Bar Plot

```
df['Age'].plot(kind='bar')
```

7. Advanced Data Operations

This involves more complex indexing, selections, and filtering techniques.

Multi-Indexing

```
arrays = [['a', 'a', 'b', 'b'], [1, 2, 1, 2]]
index = pd.MultiIndex.from_arrays(arrays, names=('letter', 'number'))
df_multi = pd.DataFrame({'data': [100, 200, 300, 400]}, index=index)
print(df_multi)
```

MultiIndexing allows you to have multiple index levels on an axis. It provides a way to work with higher dimensional data in a lower dimensional form.

Example: Creating and using a MultiIndex DataFrame.

```
import pandas as pd

# Sample data
arrays = [
    ['a', 'a', 'b', 'b'],
    [1, 2, 1, 2]
]

tuples = list(zip(*arrays))
index = pd.MultiIndex.from_tuples(tuples, names=['first', 'second'])
df = pd.DataFrame({'value': [10, 20, 30, 40]}, index=index)

print(df)

# Selecting data with MultiIndex
print(df.loc['a']) # Selecting all rows with the first index 'a'
```

Conditional Selections

```
# Select rows where 'Age' is greater than 30
print(df[df['Age'] > 30])

filtered_df = df[df['Age'] > 30] # Rows where age is greater than 30
print(filtered_df)
```

Advanced Indexing Techniques

```
# Select rows based on index position
print(df.iloc[1:3])
```

8. Data Transformation

Transforming data using functions like `apply`, and performing group operations.

Using Apply

```
# Apply a function to each element in a column
df['AgeSquared'] = df['Age'].apply(lambda x: x**2)
```

GroupBy Operations

```
# Grouping data and calculating mean for each group
grouped = df.groupby('Department')
print(grouped['Salary'].mean())
```

Pivot Tables

```
# Create a pivot table
pivot = df.pivot_table(values='Sales', index='Region', columns='Year')
print(pivot)
```

For more on data transformation, visit the [Pandas GroupBy Documentation](#).

9. Time Series Data

Pandas is excellent for manipulating time series data.

Handling Dates and Times

```
df['Date'] = pd.to_datetime(df['Date'])
print(df['Date'].dt.month) # Extract the month
```

Time-Based Indexing

```
df.set_index('Date', inplace=True)
print(df.loc['2023-01']) # Data from January 2023
```

Resampling Time Series

```
monthly_data = df.resample('M').mean()
print(monthly_data)
```

Pandas is exceptionally well suited for working with time series data, particularly with its `DateTimeIndex`.

Example: Working with `DateTimeIndex` and resampling.

```
# Creating a time series data
ts = pd.date_range('2021-01-01', periods=5, freq='D')
time_series = pd.DataFrame({'value': [10, 20, 15, 30, 45]}, index=ts)

print(time_series)

# Resampling (e.g., finding mean value for each month)
monthly_resample = time_series.resample('M').mean()
print(monthly_resample)
```

For more, explore the [Pandas Time Series/Date functionality](#).

10. Combining Data

Merging, joining, and concatenating datasets are crucial in combining data from different sources.

Get Niraj Tiwari's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

Merging Data

```
merged = pd.merge(df1, df2, on='key')
```

Joining Data

```
joined = df1.join(df2, lsuffix='_df1', rsuffix='_df2')
```

Concatenating Data

```
concatenated = pd.concat([df1, df2], axis=0)
```

Complex Merges and Joins

Pandas provides SQL-like functionalities to join or merge different datasets based on common keys.

Example: Performing an inner join and outer join.

```
# Example DataFrames
df1 = pd.DataFrame({'key': ['A', 'B', 'C'], 'value': [1, 2, 3]})
df2 = pd.DataFrame({'key': ['B', 'C', 'D'], 'value': [4, 5, 6]})

# Inner Join
```

```
inner_join = pd.merge(df1, df2, on='key', how='inner', suffixes=('_df1', '_df2'))
print(inner_join)

# Outer Join
outer_join = pd.merge(df1, df2, on='key', how='outer', suffixes=('_df1', '_df2'))
print(outer_join)
```

Check out [Merging DataFrames with Pandas](#) for more details.

11. Advanced Data Cleaning

Involves more complex techniques for cleaning data, such as dealing with outliers and missing data imputation.

Dealing with Outliers

```
# Removing outliers
Q1 = df['Salary'].quantile(0.25)
Q3 = df['Salary'].quantile(0.75)
IQR = Q3 - Q1
filter = (df['Salary'] >= Q1 - 1.5 * IQR) & (df['Salary'] <= Q3 + 1.5 * IQR)
df = df.loc[filter]
```

Complex Missing Data Imputation

```
# Fill missing values with the mean of the column
df['Salary'].fillna(df['Salary'].mean(), inplace=True)
```

12. Performance and Efficiency

Optimizing Pandas operations for better performance.

Using Categorical Data Types

```
df['Category'] = df['Category'].astype('category')
```

Optimizing Operations

```
# Using iterrows() efficiently  
for index, row in df.iterrows():  
    # Perform operations
```

Using Efficient Data Types: Pandas defaults to higher memory-consuming data types. Optimizing data types, especially for large datasets, can save memory and improve performance.

Example: Converting data types.

```
df['column'] = df['column'].astype('category') # For columns with few unique va  
df['int_column'] = pd.to_numeric(df['int_column'], downcast='integer')
```

Processing Data in Chunks: For very large datasets that do not fit into memory, you can read and process data in chunks.

Example: Processing a large CSV file in chunks.

```
chunk_size = 10000 # Adjust this according to your memory availability
for chunk in pd.read_csv('large_file.csv', chunksize=chunk_size):
    process(chunk) # Replace this with your processing function
```

Using Efficient Functions and Avoiding Loops: Vectorized operations are much faster than applying functions row-wise or using loops.

Example: Using vectorized operations over loops.

```
df['new_column'] = df['column1'] + df['column2'] # Vectorized addition
```

To learn more about optimizing Pandas performance, visit [Enhancing Performance in Pandas](#).

13. Custom Operations

Using Window Functions

```
# Rolling mean
df['RollingMean'] = df['Column'].rolling(window=3).mean()
```

Custom Aggregations with GroupBy

```
def custom_agg(x):  
    return x.max() - x.min()  
grouped = df.groupby('Category')['Value']  
grouped.agg(custom_agg)
```

Pandas allows you to define custom functions and apply them to your data, which is particularly useful for specific aggregation tasks.

Using `apply()` Function: It allows you to apply a custom function to each row/column of a DataFrame.

Example: Custom function with `apply()` .

```
def custom_func(x):  
    return x.max() - x.min()  
  
df['range'] = df.apply(custom_func, axis=1) # Applying on each row
```

Custom Aggregation with `groupby()` : For grouped data, you can use custom aggregation functions.

Example: Custom aggregation function with `groupby()` .

```
def custom_agg(x):  
    return x.quantile(0.75) - x.quantile(0.25)
```

```
grouped = df.groupby('category')['value']  
custom_agg_result = grouped.agg(custom_agg)
```

Using `applymap()` for Element-wise Operations: If you need to apply a function to each element in the DataFrame, `applymap()` is useful.

Example: Element-wise custom operation.

```
df = df.applymap(lambda x: x*2 if x > 0 else x)
```

14. Advanced Analytics and Modeling

Integration with statistical and machine learning libraries for deeper data analysis.

Statistical Analysis

Integration with Statistical Libraries: Pandas can be integrated with libraries like `statsmodels` and `scikit-learn` for advanced statistical analysis and machine learning.

Example: Linear regression using `statsmodels`.

```
import statsmodels.api as sm  
  
# Assuming df is a Pandas DataFrame with the predictor variables and target variable  
X = df.drop('y', axis=1)  
y = df['y']  
  
# Adding a constant to the model (intercept)
```

```
X = sm.add_constant(X)
model = sm.OLS(y, X).fit()
print(model.summary())
```

Machine Learning Integration

Machine Learning with Scikit-learn: For machine learning tasks, you can prepare your data using Pandas and then feed it into a scikit-learn model. ►

Example: Using Pandas DataFrames with a scikit-learn model.

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

# Splitting the dataset into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_

# Training a linear regression model
model = LinearRegression().fit(X_train, y_train)

# Making predictions and evaluating the model
predictions = model.predict(X_test)
```

15. High-Performance Pandas

Working with large datasets and optimizing performance.

Chunking Large Datasets

```
chunk_size = 10000
for chunk in pd.read_csv('large_dataset.csv', chunksize=chunk_size):
```

```
process(chunk)
```

Parallel Processing with Dask

Utilize libraries like Dask or Ray for parallel processing of large datasets.

```
# Example: Using Dask for parallel data processing.
import dask.dataframe as dd

ddf = dd.from_pandas(df, npartitions=10)
result = ddf.groupby('column').mean().compute()
```

For more on Dask, see [Dask Documentation](#).

16. Visualization and Dashboards

Creating interactive visualizations and dashboards using Pandas and other libraries.

Interactive Visualizations with Plotly

```
import plotly.express as px
fig = px.line(df, x='Date', y='Value', title='Time Series Plot')
fig.show()
```

Building Dashboards with Plotly Dash

```
import dash
import dash_core_components as dcc
import dash_html_components as html
app = dash.Dash(__name__)
app.layout = html.Div([
    dcc.Graph(figure=fig)
])
```

For more on Plotly Dash, visit [Dash Plotly](#).

17. Deep Dive into Pandas Source Code

Understanding the inner workings of Pandas can help in contributing to its development.

- Diving into the source code involves:
 - Checking out [the Pandas GitHub Repository](#).
 - Understanding Python, Cython, and C code that powers Pandas
 - Contributing to Pandas

18. Specialized Data Types

Handling more complex data types like text and categorical data.

Text Data Operations

```
# String operations
df['TextColumn'].str.upper()
df['TextColumn'].str.contains('pattern')
```

Advanced Handling of Categorical Data

```
df['Category'] = df['Category'].astype('category')  
df['Category'].cat.reorder_categories(['Low', 'Medium', 'High'], ordered=True)
```

For a deeper understanding of these advanced topics, the Pandas Documentation remains an invaluable resource. Additionally, specialized books and online courses that focus on high-level data analysis and machine learning can provide more context and applications of these Pandas features.

References

- https://pandas.pydata.org/docs/user_guide/index.html
- <https://www.kaggle.com/learn/pandas>
- <https://www.datacamp.com/courses/manipulating-dataframes-with-pandas>
- <https://pandas.pydata.org/pandas-docs/stable/>
- - MultiIndex / Advanced Indexing: https://pandas.pydata.org/pandas-docs/stable/user_guide/advanced.html
- - Time Series / Date functionality: https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html
- - Merging and Joining data: https://pandas.pydata.org/pandas-docs/stable/user_guide/merging.html

Python Programming

Pandas

Machine Learning



Written by Niraj Tiwari

44 followers · 11 following

Follow

AI | ML | DL | Large Scale Systems | Systems Thinking | Human Behaviour |
Psychology | NLP | Team Building | Leadership

No responses yet



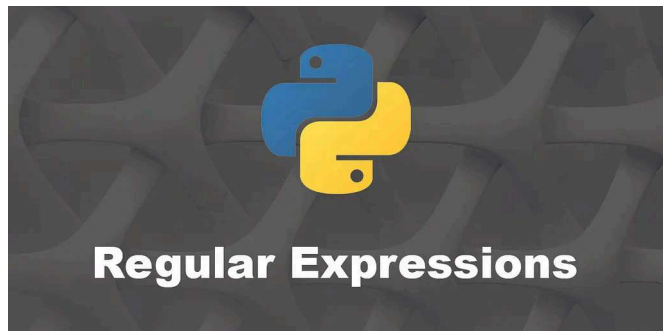
Write a response

What are your thoughts?

More from Niraj Tiwari



Niraj Tiwari



Niraj Tiwari

Unlocking the Power of BATNA and ZOPA : Transformative Strategies...

One of the most essential tools in the negotiator's toolkit is the concept of BATNA...

Jan 21, 2024



33



Niraj Tiwari

Unleashing Creativity in Team Problem Solving: Mastering TRIZ...

When it comes to innovation and solving complex problems within a team, the...

Mar 23, 2024



100



2



Python101 Regular Expression: Mastering Python RegEx in Simpl...

Regular expressions (regex) are powerful tools for searching, matching, and...

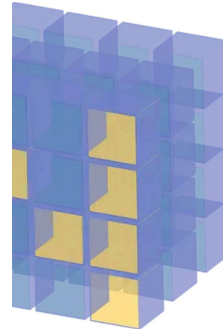
Mar 26, 2024



8



1



NumP



Niraj Tiwari

NumPy 101 : Unleash the Power of Numerical Computing in Python

Introduction to NumPy

Jan 7, 2024



8



See all from Niraj Tiwari

Recommended from Medium

:2510.01171v3 [cs.CL] 10 Oct 2025

ABSTRACT

Post-training alignment often reduces LLM diversity, leading to a phenomenon known as *mode collapse*. Unlike prior work that attributes this effect to algorithmic limitations, we identify a fundamental, pervasive data-level driver: *typicality bias* in preference data, whereby annotators systematically favor familiar text as a result of well-established findings in cognitive psychology. We formalize this bias theoretically, verify it on preference datasets empirically, and show that it plays a central role in mode collapse. Motivated by this analysis, we introduce *Verbalized Sampling (VS)*, a simple, training-free prompting strategy to circumvent mode collapse. VS prompts the model to verbalize a probability distribution over a set of responses (e.g., “Generate 5 jokes about coffee and their corresponding probabilities”). Comprehensive experiments show that VS significantly improves performance across creative writing (poems, stories, jokes), dialogue simulation, open-ended QA, and synthetic data generation, without sacrificing factual accuracy and safety. For instance, in creative writing, VS increases diversity by 1.6-2.1× over direct prompting. We further observe an emergent trend that more capable models benefit more from VS. In sum, our work provides a new data-centric perspective on mode collapse and a practical inference-time remedy that helps unlock pre-trained generative diversity.

Problem: Typicality Bias Causes Mode Collapse

Solution: Verbalized Sampling (VS) Mitigates Mode Collapse

- Path params: /items/{item_id}
- Query params: /items?skip=0&limit=10
- Request body: Pydantic models for JSON data
- Combine them in one endpoint.

Step 3 – Interactive Docs & Schema Customization
Goal: Make your API docs professional.

- Set title, description, version in FastAPI()
- Add descriptions & examples to params & models
- Define response models & status codes.

Step 4 – Async & Performance
Goal: Understand and use async for scalability.

- Use async def for I/O-bound tasks
- Avoid async for CPU-heavy tasks.

Step 5 – Dependency Injection

In Generative AI by Adham Khaled

Stanford Just Killed Prompt Engineering With 8 Words (And I...

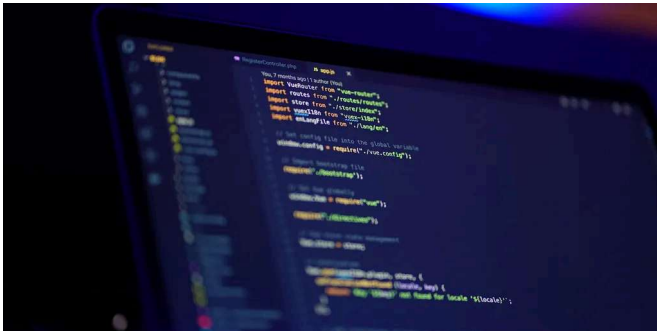
ChatGPT keeps giving you the same boring response? This new technique unlocks 2×...

🌟 Oct 19, 2025

👏 23K

💬 598

🔖



PY

In Python in Plain English by Areej Saeed

The Python GUI Library No One Talks About (But Should)

Most Python developers don't hate GUIs. They just avoid them.

🌟 4d ago

👏 24

🔖

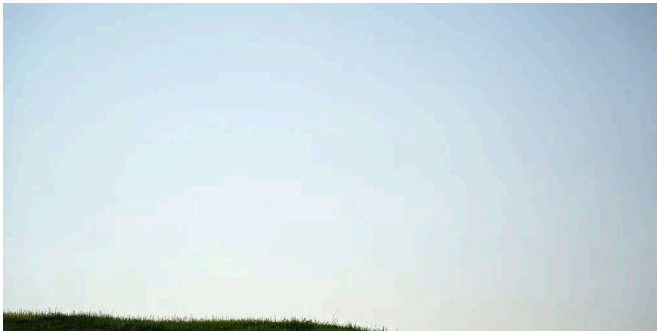
Dasamantharao Rohit

FastAPI 16-Step Cheat Sheet with Examples & Documentation Links

FastAPI is a modern, high-performance Python web framework for building APIs...

Aug 9, 2025

🔖



Elshad Karimov

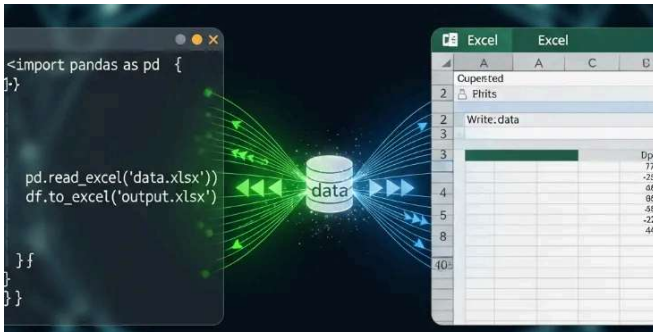
Mastering Python Generators: The Hidden Power of Lazy Evaluation

When most beginners start with Python, they focus on lists, dictionaries, and loops. But...

🌟 Sep 17, 2025

👏 5

🔖

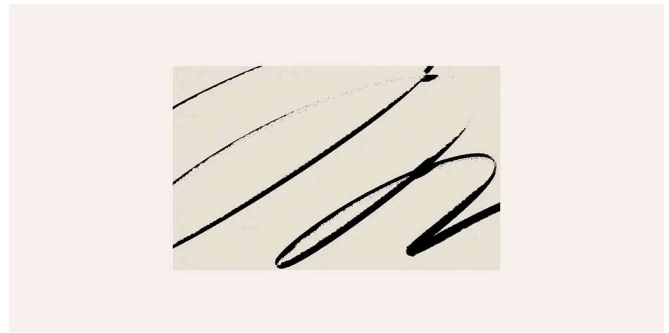


 Dr. Shouke Wei

Master Excel with Python: A Beginner's Guide to Openpyxl

A powerful, easy-to-use tutorial on how to read, write, and manipulate Excel files witho...

★ Aug 14, 2025  32



 Yi-Ning Huang

[Python] What is Numpy/ Pandas Vectorization?

 Intro:

★ Sep 5, 2025



See more recommendations